10/28 points
=> Grade: 3.6

NURa Hand-in 2

Silvan Toet (S1827332)

March 20, 2026

1 Satellite galaxies around a massive central

1.1 (1a) Determining the normalization

The spherical volume element dV can be defined as follows: We know that the volume can be computed in spherical coordinates using the triple integral

$$\iiint_V n(x)dV = \int_{\phi=0}^{2\pi}\int_{\theta=0}^{\pi}\int_{r=0}^{r_{max}} n(x)r^2 \sin\theta \, dr \, d\theta \, d\phi \tag{1}$$

$$= 4\pi A\langle N_{sat}\rangle \int_{x=0}^{x_{max}=5} x^2 \left(\frac{x}{b}\right)^{a-3} \exp(-\left(\frac{x}{b}\right)^c)dx = \langle N_{sat}\rangle \tag{2}$$

$$-> A = \frac{1}{4\pi \int_{x=0}^{x_{max}=5} x^2 \left(\frac{x}{b}\right)^{a-3} \exp(-\left(\frac{x}{b}\right)^c)dx} \tag{3}$$

Since n(x) where x is the radius relative to virial radius $x = r/r_{vir}$ is only dependent on r, we can easily solve for θ and ϕ. Dividing by $\langle N_{sat}\rangle$ on both sides and rearranging gives us an expression for A.

Next we use Romberg integration (i.e. Richardson extrapolation applied to integration) to numerically integrate the integral in the expression. Romberg integration first applies the extended trapezoid rule for N intervals, then 2N, 4N, ... up to $2^m\times$ N intervals, where m is the order of the initial approximations. Next we use the fact that each successive order's leading error term is exactly a 1/4 times the leading error term of the previous order. Subtracting the two approximations from each other in such a way as to cancel the leading error term results in m-1 new approximations. We can do this subtraction trick m-1 times. Since after each subtraction we no longer need the two values used for this, we can store the initial approximations and all subsequent improvements in a vector of length m. At the end of the algorithm the first element of the vector contains the best estimate of the integral and the best estimate for the error is the absolute difference between the first two elements. Table 1 shows the value of the normalization constant A to full precision.

Table 1: Normalization constant of the satellite galaxy profile.

Normalization constant A
9.19836296311

1.2 Code for the integrator

1a: 2.25/4

```python
##### Integrator block #####

import numpy as np


def trapezoid(a, b, func, N: int):
    """Use the extended trapezoid rule to calculate the integral.
    Parameters a and b are start and stop x values of the range to be integrated respectively.
    func is the function to be evaluated.
    N is the number of evaluations.
    """
    xdata = np.linspace(a, b, num=N)
    h = (b - a) / N  # step size
    return h * (0.5 * (func(b) + func(a)) + np.sum(func(xdata[1:-1])))


def romberg_vector_version(
    a, b, func, N_start: int, order: int = 5, return_error: bool = False
):
    """See slide 14 of Lecture 4 annotated slides for the formula used.
    N_start is the initial number of function evaluations. This will double with every order.
    order is the number of initial approximations.
    We return the best estimate for the integral stored at r_0
    If return_error is True, the error estimate abs(r_0 - r_1) is also returned
    """
    romberg_vector = np.zeros(shape=order)
    for j in range(order):
        for i in range(order - j):
            if (
                j == 0
            ):  # the first time we fill the column, we use the trapezoid rule with doubling step sizes per row entry
                N = (2**i) * N_start  # number of intervals doubles with each depth
                romberg_vector[i] = trapezoid(a, b, func, N)
            else:  # Subsequent times we use the update rule
                romberg_vector[i] = (
                    4**j * romberg_vector[i + 1] - romberg_vector[i]
                ) / (
                    4**j - 1
                )  # General version of 4/3 S_1 - 1/3 S_0
    if return_error:
        return romberg_vector[0], abs(
            romberg_vector[0] - romberg_vector[1]
        )  # (value, error)
    return romberg_vector[0]
```

1.3 (1b) Sampling the satellite distribution

We use rejection sampling to generate 10000 satellite galaxy positions. In rejection sampling we first scale the probability distribution function max value to 1. Then we generate two random numbers:

- one random number between the x ranges of our pdf. In our case this is between x_{min}=0 and x_{max}=5
- another random number between 0 and 1. This number represents the probability that the number is accepted

if y < p(x), we accept the generated x value in our sample. Since y is more likely to lie under p(x) if p(x) is large for that value of x, this incorporates the difference between probabilities for different x values. Hence the statement "2 random numbers in, <1 random number out", because we need two random numbers to generate at most 1 (we might also reject it).

We implement our random number generator as a combination of an 64-bit XOR shift method and a (Multiple) Linear Congruential Generator. Both sub RNGs receive their own starting seed, and base their subsequent generated number on only (a part of) their own previous output. The two random numbers generated by the subgenerators are then added element-wise in the combination-rng.

We could not get BigCrush our DieHard to work to thoroughly test our RNG, but we did plot a histogram of 100000 uniformly generated test values below. They seem to be quite random.

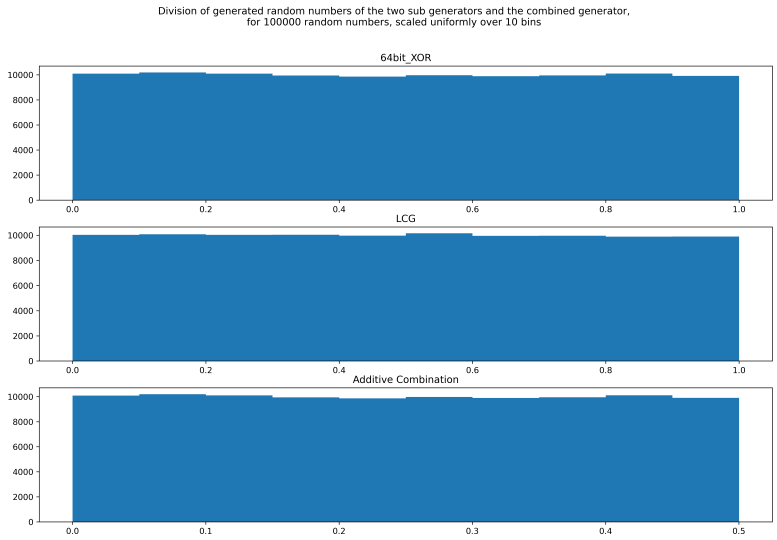

Figure 1: Comparison between the random numbers generated by the 64-bit XOR shift, MLCG and combinatory generator, for 100000 samples spread uniformly in the range (0,1) over 10 bins.

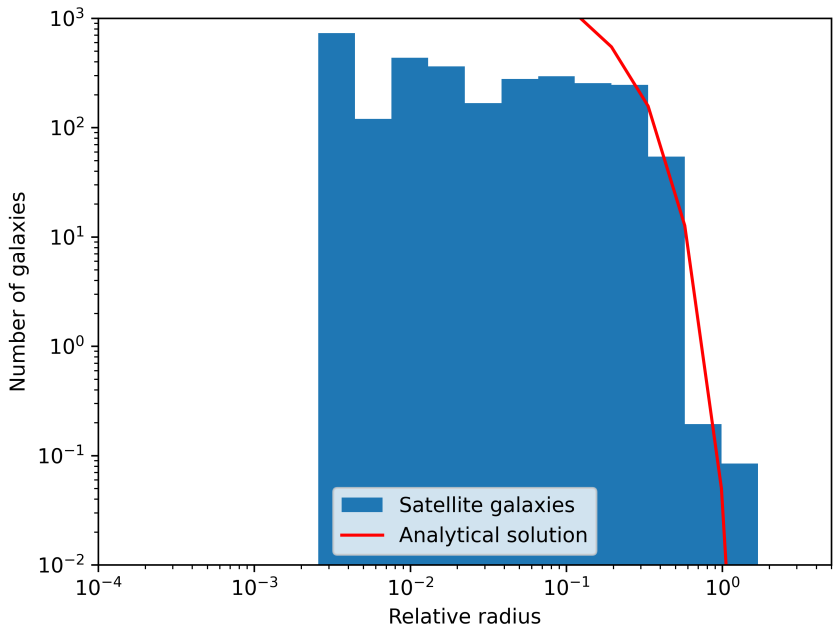

Figure 2: Comparison between the analytical satellite distribution $N(x)$ and the histogram of 10 000 sampled satellite galaxies in logarithmic bins.

we calculate the maximum value of n(x) and create a new function scaled to this maximum value. The new function serves as our probability distribution p(x) Next we plot our 10000 generated values from this probability distribution. We scale the bins with the density parameter in the histogram function to account for their varying widths on a log scale. Because this density parameter scales the integral/sum of the histograms to 1, we multiply with 100 to get the same average total number of satellites as the normalized analytical function.

As we can see, the histograms follow the analytical function around the exponential cutoff region. However, the sampler seems to undersample values at small radii.

1.4 Code for sampling

1b: 1/7

```python
#### Sampler block including RNGs ####

from collections.abc import Callable, Iterable

import numpy as np


def rng_64bit_xor_shift(
    x: np.uint64 = np.uint64(123456789),
    a_iter: Iterable[np.uint64] = (np.uint64(21), np.uint64(35), np.uint64(4)),
    size: int = 1,
    scale_uniform: bool = False,
):
```

```python
    """Use the 64-bit XOR-shift method for making a Random Number Generator on slide 14 of lecture 5.
    x is the seed, which should be an unsigned 64 bit integer, not 0.
    We use a numpy variable, as Python built-in variables will automatically extend if bits are moved out of range (which is what we want to use here: moving bits out of range hides info for the user).
    a is an iterable of ints containing the coefficients that will be used to bitshift x by a certain amount before doing an XOR operation with itself.
    Size is the number of random numbers generated. scale_uniform scales the output to the range [0,1]
    If scale_uniform is False, the output is in the range (0, 2^32-1), since we only supply the lowest 32 bits of each generated random number to the user to hide info about the state of the RNG.
    """
    if x == 0:
        x = 135797531
    x = np.uint64(x)
    a_list = [
        np.uint64(a) for a in a_iter
    ]  # Convert all coefficients to np.uint64 variables once, to ensure no error occurs if the user does not supply these in np.uint64 format
    if x == 0:
        x = 135797531

    random_values_array = np.zeros(size, dtype=np.uint32)
    for i in range(len(random_values_array)):
        for j, a in enumerate(a_list):
            x = (
                x ^ (x >> a) if j % 2 == 0 else x ^ (x << a)
            )  # Even coefficients are used to shift x a bits to the right, odd coefficients are used to shift x a bits to the left
            # This is necessary to ensure randomness: otherwise we would only be changing e.g. the lower significant bits, and numbers would oscillate around the starting seed value
        random_values_array[i] = np.uint32(
            x
        )  # We use a bitmask to provide the user with only the lowest 32 bits: restrict information about the full state to the user!
    if scale_uniform:
        return (
            random_values_array / np.iinfo(np.uint32).max
        )  # Divide by max np.int32 value because we only supply the lowest 32 bits to the user.
    # Division operator converts dtypes to np.float64, which is what we want here
    return random_values_array


def lcg(
    x: np.uint64 = np.uint64(123456789),
    a: np.uint64 = np.uint64(7654321),
    c: np.uint64 = np.uint64(3333333),
    m: np.uint64 = np.uint64(1234543212345),
    size: int = 1,
    scale_uniform: bool = False,
):
    """Implement a (Multiple) Linear Congruential Generator.
    When c=0 we are dealing with a MLCG.
    This RNG is bad on its own but a good building block for more complex RNGs.
    Size is the number of random numbers generated. scale_uniform scales the output to the range [0,1]
    If scale_uniform is False, the output is in the range (0, 2^32-1), since we only supply the lowest 32 bits of each generated random number to the user to
```

```python
    hide info about the state of the RNG.
    """
    if x == 0:
        x = 135797531
    x, a, c, m = (
        np.uint64(x),
        np.uint64(a),
        np.uint64(c),
        np.uint64(m),
    )  # Cast to unsigned integers should user forget to do so
    random_values_array = np.zeros(size, dtype=np.uint32)
    for i in range(len(random_values_array)):
        x = (a * x + c) % m
        random_values_array[i] = np.uint32(x)
    if scale_uniform:
        return random_values_array / np.iinfo(np.uint32).max
    return random_values_array


def additive_combined_rng(
    x1: np.uint64 = np.uint64(123456789),
    x2: np.uint64 = np.uint64(987654321),
    a_iter: Iterable[np.uint64] = (np.uint64(21), np.uint64(35), np.uint64(4)),
    a: np.uint64 = np.uint64(7654321),
    c: np.uint64 = np.uint64(3333333),
    m: np.uint64 = np.uint64(1234321),
    size: int = 1,
    scale_uniform: bool = True,
):
    """Combine two RNGs together. In this case a 64 bit XOR shift RNG and a Linear Congruential Generator.
    Default values have been set to values that seem to generate random results.
    """
    P_64 = rng_64bit_xor_shift(
        x=x1, a_iter=a_iter, size=size, scale_uniform=scale_uniform
    )
    P_lcg = lcg(x=x2, a=a, c=c, m=m, size=size, scale_uniform=scale_uniform)
    P_add = P_64 + P_lcg
    if scale_uniform:
        return (
            P_add / 2
        )  # Both sub generator outputs have been scaled to [0,1) range, so here we need to divide by 2
    return P_add


def sampler(dist: Callable, a: float, b: float, Nsamples: int) -> np.ndarray:
    """Sample from a normalized distribution using rejection sampling.

    Parameters
    ----------
    dist : callable
        Distribution to sample
    a :
        Minimum value for sampling
    b : float
        Maximum value for sampling
    Nsamples : int
        Number of samples

    Returns
    -------
```

```python
    sample: ndarray
        Values sampled from dist, shape (Nsamples,)
    """
    samples = np.zeros(Nsamples)
    number_of_acquired_samples = 0
    total_generated_samples = 0
    x1, x2, x3, x4 = 123456789, 987654321, 555555, 777777
    while (
        number_of_acquired_samples < Nsamples
    ):  # Since we might reject a lot of samples, keep generating samples in batches of Nsamples until we have enough
        P1_uniform = additive_combined_rng(
            np.uint64(x1), np.uint64(x2), size=Nsamples
        )  # Generate random numbers for the abscissa range of the distribution that we to sample from
        P_ab = a + (b - a) * P1_uniform  # Scale these to the abscissa range
        P2_uniform = additive_combined_rng(
            np.uint64(x3), np.uint64(x4), size=Nsamples
        )  # Then generate a separate random number from a uniform distribution [0,1), which we interpret as the probability
        for x, y in zip(  # noqa: B905
            P_ab, P2_uniform
        ):  # Then we accept x into our sample if y < p(x): the higher p(x) is for that value of x, the more likely x should be, and therefore the more likely that y is smaller than p(x)
            total_generated_samples += 1
            if y < dist(x):
                samples[number_of_acquired_samples] = x
                number_of_acquired_samples += 1
            if number_of_acquired_samples >= Nsamples:
                break
        x1, x2, x3, x4 = (
            P1_uniform[-2],
            P1_uniform[-1],
            P2_uniform[-2],
            P2_uniform[-1],
        )  # Set seeds for next batch generation to last generated values, otherwise they will start with the same seed!

    return samples
```

1.5 (1c) Selecting and sorting satellites

We are asked to select 100 galaxies with equal probability, without duplicates and without rejection of a generated sample. As explained during the lecture, a Linear Congruential Generator is not a great RNG on its own, partially due to the fact it cannot generate duplicate samples before it starts to repeat itself. Once it starts to repeat itself, the entire sequence repeats (becomes periodic). But this means that if we choose our LCG parameters (a,c and m) right, we can ensure that we get 100 random numbers with equal probability without a duplicate! A quick search on LCGs on wikipedia (`https://en.wikipedia.org/wiki/Linear_congruential_generator`) reveals that if we satisfy the Hull-Dobell theorem, we are guaranteed to get a full sequence of period m for every seed value x. The three criteria to be met are:

1. m and c are comprime
2. a - 1 is divisible by all prime factors of m
3. a - 1 is divisible by 4 if m is divisible by 4

We choose m to be the size of the array that we sample from, in our case 10000 which is the

number of initally generated satellite galaxies. Next we choose a and c to satisfy the above criteria. m has as prime factors 2 and 5. So choose c as odd and not divisible by 5. a must be divisible by 2, 4 and 5. We choose a=21 and c=37. The generated indices binned are shown in figure 3.

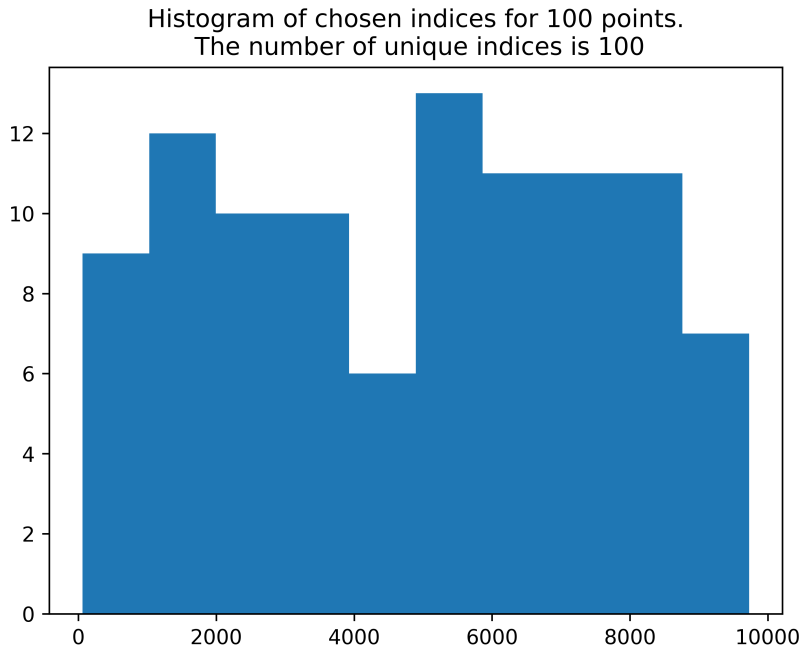

Figure 3: Binned generated indices. The title figure contains a check to see if there are indeed 100 unique indices. Since there are only a 100 generated values, the distribution does not look as flat as the other generated samples.

Now we sort the 100 selected galaxies using our sorting algorithm. Our first attempt was to use our quicksort algorithm. The quicksort algorithm avoids the pitfall of simple sorting techniques such as selection sort that need N^2 comparisons for an array of length N. It does this by splitting the array into smaller and smaller subarrays and sorting these recursively. We first use selection sort to choose the pivot element from the first, middle and last element. Then we start swapping elements to the left of the pivot that are larger than the pivot with elements to the right of the pivot that are smaller than the pivot. This ensures that after 1 recursion, we end up with two subarrays that, although they are still unsorted themselves, their elements are in the correct position compared to the pivot element. Doing this recursively eventually sorts the entire array with an order O(N log N) comparisons.

Unfortunately, checking the array after the quicksort call revealed there were some values not sorted correctly. We therefore resort to our own selection sort as a fallback option for this assignment. We have left both algorithms in the code for inspection and will be very happy if any of the graders can spot the mistake in the quicksort algorithm.

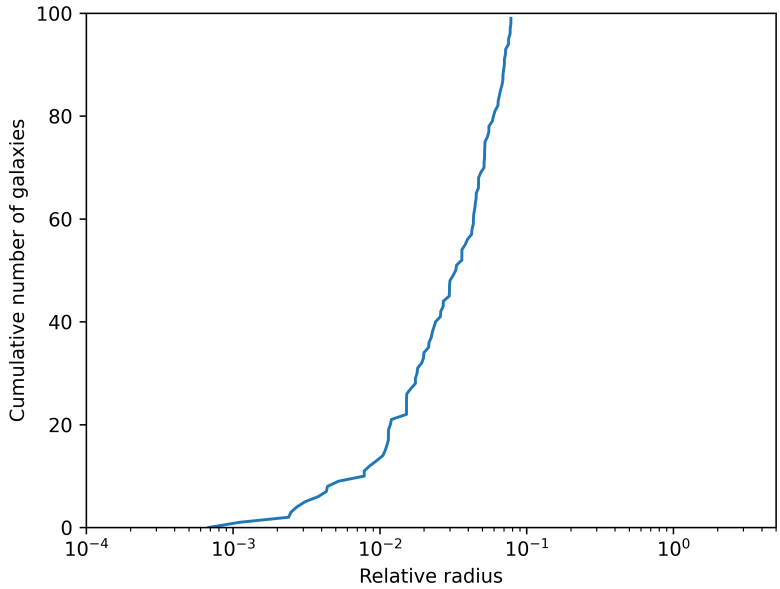

Figure 4: Cumulative number of selected galaxies as a function of relative radius.

1.6 Code for selection and sorting

```
#### Sorting and Selection block ####


from collections.abc import MutableSequence
from copy import deepcopy

import numpy as np
from Q1_satellites_sampling import lcg


def swap(a, i, j):
    """Helper function to swap elements a[i] and a[j]"""
    a[i], a[j] = a[j], a[i]


def selection_sort(A: MutableSequence, return_index: bool = False):
    """Implement the selection sorting algorithm.
    Based on Lecture 6 slide 6.
    If return_index is True, the sorted index array is returned.
    """
    a = deepcopy(A)
    N = len(a)
    if return_index:
        index_a = np.arange(N)
    for i in range(N - 1):
        i_min = i
        for j in range(i + 1, N):
            if a[j] < a[i_min]:
                i_min = j
```

```
        if i_min != i:
            swap(a, i, i_min)
            if return_index:
                swap(index_a, i, i_min)
    if return_index:
        return a, index_a
    return a


def quicksort(A: MutableSequence, return_index: bool = False):
    """Implement the quicksort sorting algorithm.
    Based on Lecture 6 slide 9.
    """
    a: MutableSequence = deepcopy(A)
    N = len(a)
    start_indx_fml = [0, N // 2, N - 1]
    if return_index:
        index_a = np.arange(N)
    first_middle_last, indx_fml = selection_sort(
        [a[start_indx_fml[0]], a[start_indx_fml[1]], a[start_indx_fml[2]]],
        return_index=True,
    )  # take the median of the first, last and middle element as the pivot
    a[start_indx_fml[0]], a[start_indx_fml[1]], a[start_indx_fml[2]] = (
        first_middle_last[0],
        first_middle_last[1],
        first_middle_last[2],
    )
    if return_index:
        (
            index_a[start_indx_fml[0]],
            index_a[start_indx_fml[1]],
            index_a[start_indx_fml[2]],
        ) = (indx_fml[0], indx_fml[1], indx_fml[2])

    def recursive_part(sub_a: MutableSequence, sub_start_index: int):
        """This inner function is called recursively to sort smaller and smaller subarrays.
        sub_start_index tells the inner function where this specific subarray started, and therefore where its sorted values should be placed in the total array.
        """
        N = len(sub_a)
        x_pivot = sub_a[
            N // 2
        ]  # since N will become smaller once we apply this step recursively, we don't hardcode this
        i = 0
        j = N - 1

        conda, condb = (
            sub_a[i] >= x_pivot,
            sub_a[j] <= x_pivot,
        )  # Define these conditions as variables to avoid evaluating them unnecessarily many times!
        if (
            conda and condb
        ):  # Conditions might already be True in initial state, swap if so
            swap(sub_a, i, j)
            conda, condb = False, False
        while (not conda) or (not condb):
            # i goes up from 0 until a[i]>=x_pivot
            if not conda:
```

```
                i += 1
                conda = sub_a[i] >= x_pivot
            # j goes down from N-1 until a[j]<=x_pivot
            if not condb:
                j -= 1
                condb = sub_a[j] <= x_pivot
            # pointers have crossed. If we don't check this we will undo all our hard work: the elements that we have swapped will be swapped back. Also, size -2 subarrays will flip wrongly!
            if j <= i:
                break
            # once we meet both conditions, swap the elements and set conditions to false to continue this iteration of the loop
            if conda and condb:
                swap(sub_a, i, j)
                conda, condb = False, False

        sub_a_low, sub_a_high = sub_a[0:i], sub_a[i:N]
        if len(sub_a_low) == 1 and len(sub_a_high) == 1:
            # put at right position in final array
            a[sub_start_index : sub_start_index + 1] = sub_a_low
            a[sub_start_index + 1 : sub_start_index + 2] = sub_a_high
        elif len(sub_a_low) == 1:
            a[sub_start_index : sub_start_index + 1] = sub_a_low
            recursive_part(sub_a_high, sub_start_index + 1)
        elif len(sub_a_high) == 1:
            a[sub_start_index + i : sub_start_index + i + 1] = sub_a_high
            recursive_part(sub_a_low, sub_start_index)
        else:
            recursive_part(sub_a_low, sub_start_index), recursive_part(
                sub_a_high, sub_start_index + i
            )
        return sub_a

    a = recursive_part(a, 0)
    return a


def choice(array_size: int, number_of_samples: int = 1) -> np.ndarray:
    """Choose given number of random indices for an array, without replacement
    We use the Hull-Dobell theorem for Linear Congruential Generators, which ensures all values up to the period m are generated exactly once before the sequence repeats.
    Note that the set values for a and c have been set manually to satisfy Hull-Dobell for an array of size 10000.

    Parameters
    ----------
    arr : ndarray
        Array to shuffle
    size : int, optional
        Number of elements to pick from array
        The default is 1

    Returns
    -------
    chosen : ndarray
        Randomly chosen elements from arr, shape (size,)
    """
    return lcg(a=21, c=37, m=array_size, size=number_of_samples)
```

1.7 (1d) Numerical derivative

Table 2: Derivative of the satellite number density at $x = 1$.

Method	Value
Analytical	-0.62556708476
Numerical	-0.897338413293

Now we compare the analytical derivative of n(x), namely dn(x)/dx, to the numerical derivative. Our plan was to use Ridder's method to accurately compute the numerical derivative. Ridder's method applies Richardson extrapolation to differentiation. We first generate m approximations with decreasing step sizes of h using the central difference method. Then we use the update rule

$$D_{i,j} = \frac{d^{2j} D_{i+1,j-1} - D_{i,j-1}}{d^{2j} - 1} \tag{4}$$

To cancel out error terms shared by the approximations. This builds a piramide, or more efficiently, a vector, where the first element holds the best approximation of the derivative.

Unfortunately we could not get Ridder's method to work for the numerical derivative. We therefore use the central difference method. We have included our code for ridder, both in the form for a single x and for the case where x is an array of abscissa values. We would appreciate feedback on this as well.

1.8 Code for derivative calculation

```
##### Derivative block #####


from collections.abc import Callable
from copy import deepcopy

import numpy as np


def dn_dx(
    x: np.ndarray, A: float, Nsat: float, a: float, b: float, c: float
) -> np.ndarray:
    """Analytical derivative of number density provide

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
```

```python
    -------
    float | ndarray
        Same type and shape as x. Derivative of number density of
        satellite galaxies at given radius x.
    """
    return (
        A
        * Nsat
        * b**3
        * (x / b) ** a
        * np.exp(-((x / b) ** c))
        * (-3 + a - c * (x / b) ** c)
        / x**4
    )


def central_difference(func: Callable, h: float, x: np.ndarray) -> float:
    return func(x + h) - func(x - h) / (2 * h)


def ridder(
    func: Callable,
    h: float,
    x: np.ndarray,
    d: float = 2,
    m: int = 5,
    target_error: float = (np.finfo(float).eps) ** 0.8,
):  # noqa: RET503
    """Use Ridder's method (Richardson extrapolation for differentiation) to compute the numerical derivative of a function func,
    using the central differences formula with an initial h.
    d is the factor to decrease h with for successive approximations.
    m is the number of initial approximations.
    This approach is based on slide 8 of lecture 5.
    We will continue improving our estimate until the best estimate is smaller than the target error.
    The default target error is the machine relative error to the power 0.8, based on slide 9 of lecture 5.
    """
    d_inv = 1 / d  # calculate this once for efficiency
    h_vector = np.zeros(10 * m)
    improvement = 2 * target_error  # make sure while loop starts
    best_approx = 2**64
    j = 0
    while True:
        j += 1
        for i in range(m - j):
            if j == 0:
                h_vector[i] = central_difference(func, h, x)
                h *= d_inv
            else:
                # Now combine pairs of approximations (step 3 of slide 8), but with update rule adapted for 1D version (overwriting of previous approximations)
                h_vector[i] = (d ** (2 * j) * h_vector[i + 1] - h_vector[i]) / (
                    d ** (2 * j) - 1
                )
        improvement = abs(best_approx - h_vector[0])
        if (
            improvement < target_error
        ):  # Terminate when improvement over previous best approx is smaller than target error
```

```python
            return best_approx
        # if improvement > previous_improvement:  # How can I measure if error grows? like this?
        #     return best_approx
        # previous_improvement = improvement


def ridder_array_method(
    func: Callable,
    h: float,
    x: np.ndarray,
    d: float = 2,
    m: int = 5,
    target_error: float = (np.finfo(float).eps) ** 0.8,
):  # noqa: RET503
    """Use Ridder's method (Richardson extrapolation for differentiation) to compute the numerical derivative of a function func,
    using the central differences formula with an initial h.
    d is the factor to decrease h with for successive approximations.
    m is the number of initial approximations.
    This approach is based on slide 8 of lecture 5, in such a way that it can be called with an x array at once.
    We will continue improving our estimate until the best estimate is smaller than the target error.
    The default target error is the machine relative error to the power 0.8, based on slide 9 of lecture 5.
    """
    D = np.zeros(
        (m, len(x))
    )  # matrix storing m solution approximations for each x value in the input x array

    d_inv = 1 / d  # calculate this once for efficiency
    h_vector = np.zeros(m)
    h_vector[0] = h  # for plotting purposes, remove when we want only efficiency.
    improvement = 2 * target_error  # make sure while loop starts
    for j in range(m):
        for i in range(m - j):
            best_approx = deepcopy(D[0])
            if j == 0 and i == 0:
                D[0] = central_difference(func, h, x)
            elif j == 0:
                h *= d_inv
                h_vector[i] = h
                D[i] = central_difference(func, h, x)
            else:
                # Now combine pairs of approximations (step 3 of slide 8), but with update rule adapted for 1D version (overwriting of previous approximations)
                D[i] = (d ** (2 * j) * D[i + 1] - D[i]) / (d ** (2 * j) - 1)
            improvement = abs(best_approx - D[0])
            if (
                improvement < target_error
            ):  # Terminate when improvement over previous best approx is smaller than target error
                return D[0], h_vector
            # if improvement > previous_improvement:  # How can I measure if error grows? like this?
            #     return best_approx
            # previous_improvement = improvement
```

2 Heating and cooling in HII regions

2.1 (2a) Photoionization heating and radiative recombination cooling

root finding deals with finding the point x on the abscissa where a function f(x) crosses it e.g. f(x) = 0. This can also be used to find intersections (in this case the equilibrium temperature) of two functions, simply by defining the difference function h(x) = g(x) - f(x). We decided to compare three different root finding methods: bisection, secant and false position. In all three methods we start with a function and a bracket [a,b] in which a single root must lie.

This can be determined by checking if f(a) * f(b) ¡ 0: one function value must be positive and one negative and therefore their product negative. Bisection takes a new point c halfway between a and b and evaluates the function there. If the root is between a and c, this becomes the new interval. Same for b and c. a,b and c are updated in subsequent steps.

Secant determines a new point c by drawing a line from a to b and looking where it hits the x axis. again, a,b and c are updated in subsequent steps. However, this method is prone to divergence: the new point c could land outside the original bracket.

False position does essentially the same as secant, but also keeps the counterpoint (the previous best estimate). It then checks if the new point would fall outside the bracket, and if so, use the counterpoint.

Secant and false position can in theory converge faster than bisection, but can get stuck on function that suddenly start to grow/shrink rapidly. It is always a good idea to first examine our function.

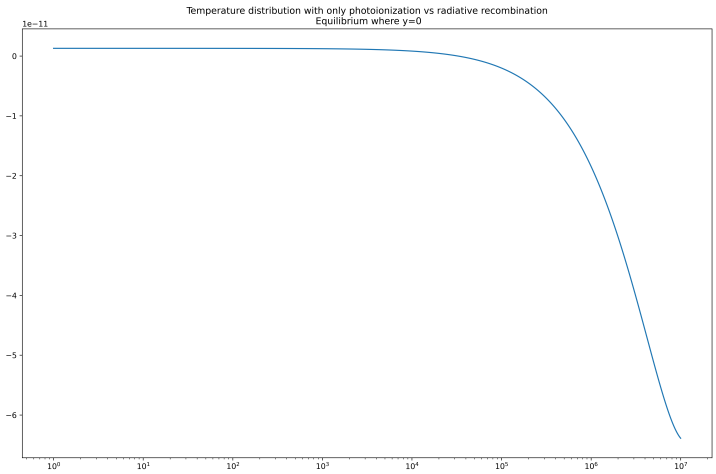

Figure 5: Equilibrium temperature function 1.

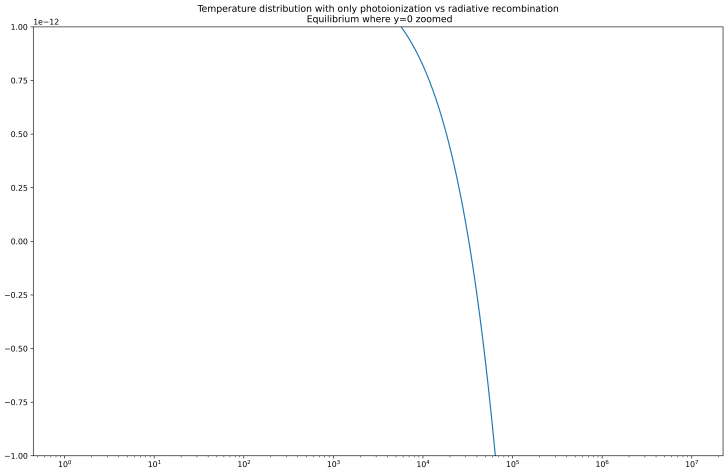

Figure 6: Equilibrium temperature function 1, zoomed in on the root.

Table 3: Equilibrium temperature for the simplified heating–cooling model for different root finding methods.

Temperature [K] and Number of iterations	
bisection	`root 32349.6295776, n_iter 14`
secant	`root nan, n_iter 50`
false position	`root 32755.2554252, n_iter 4`

The function is indeed one likely for secant and false position to get stuck on. As we can see from the table and figures, bisection and false position do quite well. False position converges faster. Secant diverges and therefore returns nan.

2.2 (2b) More realistic model

We again first examine our function, for different values of n_H. We then apply bisection, secant and false position again.

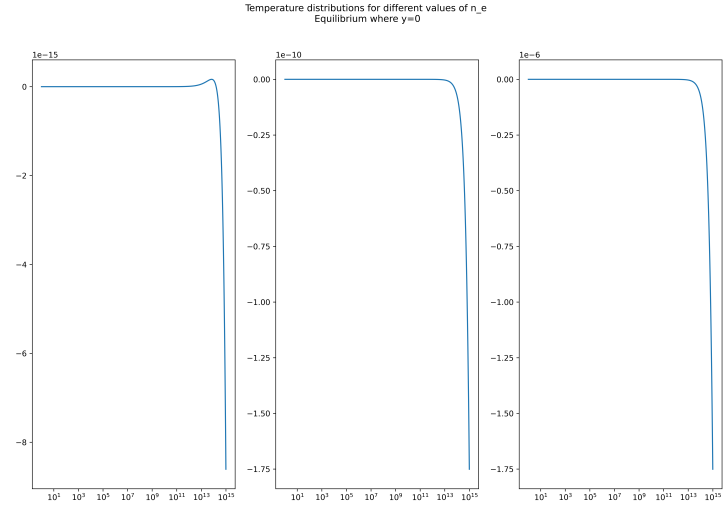

Figure 7: Equilibrium temperature function 2.

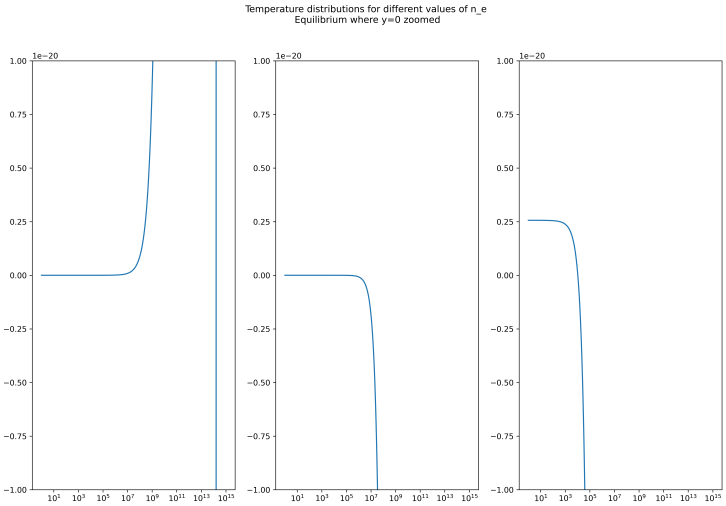

Figure 8: Equilibrium temperature function 2, zoomed in on the roots.

Table 4: Equilibrium temperature for different gas densities for bisection method.

Density n_H [cm^{-3}]	Equilibrium temperature [K] and number of iterations
10^{-4}	`root 1.60648345947e+14, n_iter 18`
1	`root 29802323.3877, n_iter 25`
10^4	`root 29104.8304567, n_iter 35`

Table 5: Equilibrium temperature for different gas densities for secant method.

Density n_H [cm^{-3}]	Equilibrium temperature [K] and number of iterations
10^{-4}	`root 58085.875, n_iter 1`
1	`root 5.375, n_iter 1`
10^4	`root 2.5, n_iter 1`

Table 6: Equilibrium temperature for different gas densities for false position method.

Density n_H [cm^{-3}]	Equilibrium temperature [K] and number of iterations
10^{-4}	`root 58085.875, n_iter 1`
1	`root 5.375, n_iter 1`
10^4	`root 2.5, n_iter 1`

As we can see from the figures and tables, bisection performes the best by far: it finds the root in all three cases. Secant and false position fail miserably, being off by many orders of magnitude.

2.3 Code for Exercise 2

```python
from collections.abc import Callable
from functools import partial
from matplotlib import pyplot as plt
import numpy as np

# Constants (mind the units!)

psi = 0.929
Tc = 1e4  # K
Z = 0.015
k = 1.38e-16  # erg/K
aB = 2e-13  # cm^3 / s
A = 5e-10
xi = 1e-15


# There's no need for nH nor ne as they cancel out
def equilibrium1(T, Z, Tc, psi):
    return (
        psi * Tc * k - (0.684 - 0.0416 * np.log(T / (1e4 * Z * Z))) * T * k
    )  # Note: this is already Gamma_pe - Lambda_rr for the first question, with n_h and n_e cancelled out.


# Therefore we can equate this to 0 and simply find the zero-crossing aka root.


def equilibrium2(T, Z, Tc, psi, nH, A, xi, aB):
    return (
        (
            psi * Tc
            - (0.684 - 0.0416 * np.log(T / (1e4 * Z * Z))) * T
            - 0.54 * (T / 1e4) ** 0.37 * T
        )
        * k
        * nH
```

```python
        * aB
        + A * xi
        + 8.9e-26 * (T / 1e4)
    )  # and here the same applies


# Derivative function, might be useful if using Newton-Raphson method for root finding
# def equilibrium2_deriv(T, nH):
#     # TODO: Compute derivative of equilibrium2 with respect to T
#     return 0.0


#### root finder ####


def bisection(
    func: Callable,
    a: float,
    b: float,
    abs_err: float,
    rel_err: float,
    max_number_of_iterations: int = 50,
):
    """Implement the bisection algorithm for finding a single root for a 1D function.
    a and b are the boundaries of the interval in which the root should lie.
    """
    fa, fb = func(a), func(b)
    if (
        not fa * fb < 0
    ):  # if the root is inside the bracket, one function evaluation is positive and the other is negative, so the product must be negative!
        raise ValueError(
            "Product of function evaluations not negative. If the root is inside the bracket, one function evaluation is positive and the other is negative, so the product must be negative!"
        )

    fc = -1
    iteration = 0
    while (
        not np.isclose(fc, 0.0, rtol=rel_err, atol=abs_err)
        and iteration < max_number_of_iterations
    ):
        c = (a + b) * 0.5
        fc = func(c)  # middle function evaluation
        if fc * fa < 0:  # root is in between low and middle value
            fb = fc  # update function evaluation of high value to middle value
            b = c  # update bracket for next c calculation
        else:  # root is in between middle and high value
            fa = fc  # update function evaluation of low value to middle value
            a = c
        iteration += 1
    return c, iteration


def secant(
    func: Callable,
    a: float,
    b: float,
    abs_err: float,
```

```python
    rel_err: float,
    max_number_of_iterations: int = 50,
):
    """Implement the secant algorithm for finding a single root for a 1D function.
    a and b are the boundaries of the interval in which the root should lie.
    """
    fa, fb = func(a), func(b)
    if (
        not fa * fb < 0
    ):  # if the root is inside the bracket, one function evaluation is positive and the other is negative, so the product must be negative!
        raise ValueError(
            "Product of function evaluations not negative. If the root is inside the bracket, one function evaluation is positive and the other is negative, so the product must be negative!"
        )

    fc = -1
    iteration = 0
    while (
        not np.isclose(fc, 0.0, rtol=rel_err, atol=abs_err)
        and iteration < max_number_of_iterations
    ):
        c = b - ((b - a) / (fb - fa)) * fb
        # if ((a - c) * (c - b)) < 0:  # a should be smaller than c and c should be smaller than b while inside bracket, so both negative, so product positive
        #    raise ValueError("Error: Secant jumped out of bracket")
        fc = func(c)  # middle function evaluation
        iteration += 1
        b, a = (
            c,
            b,
        )  # update b and a (x_i and x_i-1) to c and b (x_i+1 and x_i) respectively
        fa, fb = func(a), func(b)
    return c, iteration


def false_position(
    func: Callable,
    a: float,
    b: float,
    abs_err: float,
    rel_err: float,
    max_number_of_iterations: int = 50,
):
    """Implement the false position algorithm for finding a single root for a 1D function.
    a and b are the boundaries of the interval in which the root should lie.
    """
    fa, fb = func(a), func(b)
    if (
        not fa * fb < 0
    ):  # if the root is inside the bracket, one function evaluation is positive and the other is negative, so the product must be negative!
        raise ValueError(
            "Product of function evaluations not negative. If the root is inside the bracket, one function evaluation is positive and the other is negative, so the product must be negative!"
        )

    fc = -1
```

```python
    iteration = 0
    while (
        not np.isclose(fc, 0.0, rtol=rel_err, atol=abs_err)
        and iteration < max_number_of_iterations
    ):
        c = b - ((b - a) / (fb - fa)) * fb
        fc = func(c)  # middle function evaluation
        if fa * fc < 0:  # root between a and c
            b = c
            fb = func(b)
        else:  # root between b and c
            b, a = c, b
            fa, fb = func(a), func(b)
        iteration += 1
    return c, iteration


def main():
    equilibrium1_partial = partial(equilibrium1, Z=Z, Tc=Tc, psi=psi)
    equilibrium2_partial = partial(equilibrium2, Z=Z, Tc=Tc, psi=psi, A=A, xi=xi, aB=aB)
    # Initial bracket
    bracket = (1, 1e7)

    x1_range = 10 ** np.linspace(np.log10(bracket[0]), np.log10(bracket[1]), num=1000)

    plt.figure(figsize=(16, 10))
    plt.title(
        "Temperature distribution with only photoionization vs radiative recombination\n Equilibrium where y=0"
    )
    plt.plot(x1_range, equilibrium1_partial(x1_range))
    plt.semilogx()
    plt.savefig("Plots/equilibrium1.png", dpi=600)

    plt.figure(figsize=(16, 10))
    plt.title(
        "Temperature distribution with only photoionization vs radiative recombination\n Equilibrium where y=0 zoomed"
    )
    plt.plot(x1_range, equilibrium1_partial(x1_range))
    plt.ylim(-1e-12, 1e-12)
    plt.semilogx()
    plt.savefig("Plots/equilibrium1_zoomed.png", dpi=600)

    algorithm_list = [bisection, secant, false_position]

    with open("Calculations/equilibrium_temp_simple.txt", "w") as f:
        for algorithm in algorithm_list:
            root, number_of_iterations = algorithm(
                equilibrium1_partial,
                x1_range[0],
                x1_range[-1],
                abs_err=1e-14,
                rel_err=1e-12,
            )
            f.write(f"root {root:.12g}, n_iter {number_of_iterations}\n")

    ### 2b ####

    # Initial bracket
```

```
    bracket = (1, 1e15)
    x2_range = 10 ** np.linspace(np.log10(bracket[0]), np.log10(bracket[1]), num
    =1000)

    ne_list = [1e-4, 1, 1e4]

    fig2a, axs = plt.subplots(1, 3, figsize=(16, 10))
    plt.suptitle(
        "Temperature distributions for different values of n_e\n Equilibrium where
     y=0"
    )
    for i, ne in enumerate(ne_list):
        axs[i].plot(x2_range, equilibrium2_partial(T=x2_range, nH=ne))
        axs[i].set_xscale("log")
    plt.savefig("Plots/equilibrium2.png", dpi=600)

    fig2a, axs = plt.subplots(1, 3, figsize=(16, 10))
    plt.suptitle(
        "Temperature distributions for different values of n_e\n Equilibrium where
     y=0 zoomed"
    )
    for i, ne in enumerate(ne_list):
        axs[i].plot(x2_range, equilibrium2_partial(T=x2_range, nH=ne))
        axs[i].set_xscale("log")
        axs[i].set_ylim(-1e-20, 1e-20)
    plt.savefig("Plots/equilibrium2_zoomed.png", dpi=600)

    # We again assume the gas is fully ionized, so for nH we use the given values
    for ne
    for nH in ne_list:
        print()
        for algorithm in algorithm_list:
            equilibrium2_with_nh = partial(equilibrium2_partial, nH=nH)
            root, number_of_iterations = algorithm(
                equilibrium2_with_nh,
                x2_range[0],
                x2_range[-1],
                abs_err=1e-14,
                rel_err=1e-12,
            )
        if nH == 1e-4:
            with open("Calculations/equilibrium_low_density.txt", "w") as f:
                for algorithm in algorithm_list:
                    equilibrium2_with_nh = partial(equilibrium2_partial, nH=nH)
                    root, number_of_iterations = algorithm(
                        equilibrium2_with_nh,
                        x2_range[0],
                        x2_range[-1],
                        abs_err=1e-20,
                        rel_err=1e-12,
                    )
                    f.write(f"root {root:.12g}, n_iter {number_of_iterations}\n")
        elif nH == 1:
            with open("Calculations/equilibrium_mid_density.txt", "w") as f:
                for algorithm in algorithm_list:
                    equilibrium2_with_nh = partial(equilibrium2_partial, nH=nH)
                    root, number_of_iterations = algorithm(
                        equilibrium2_with_nh,
                        x2_range[0],
                        x2_range[-1],
                        abs_err=1e-20,
                        rel_err=1e-12,
```

```
                    )
                    f.write(f"root {root:.12g}, n_iter {number_of_iterations}\n")
        elif nH == 1e4:
            with open("Calculations/equilibrium_high_density.txt", "w") as f:
                for algorithm in algorithm_list:
                    equilibrium2_with_nh = partial(equilibrium2_partial, nH=nH)
                    root, number_of_iterations = algorithm(
                        equilibrium2_with_nh,
                        x2_range[0],
                        x2_range[-1],
                        abs_err=1e-20,
                        rel_err=1e-12,
                    )
                    f.write(f"root {root:.12g}, n_iter {number_of_iterations}\n")


if __name__ == "__main__":
    main()
```

3 Conclusions

This is a short summary as I have moved the main discussion of the results to each subquestion. In the first exercise we first integrated a satellite galaxy distribution using Romberg integration. We then generated 10000 satellites from the probability distribution using rejection sampling and our own combined RNG, made up of an 64 bit XOR shift and Linear Congruential Generator. We compared the generated values to the analytical distribution. Next we randomly selected 100 galaxies without duplicates or rejection using an LCG with a full period using the Hull-Dobell theorem. Then we sorted these: first we tried our implementation of quicksort. When this failed we resorted to our implementation of heapsort. Finally we took the numerical derivative of the distribution and compared this to the analytical distribution. We first tried Ridder's method for derivation. When this failed we resorted to central differences.

In the second exercise we used three different implementations of root finding algorithms, namely bisection, secant and false position, to find the roots of two equilibrium temperature functions of HII regions. Due the rapidly decreasing nature of the functions, bisection performed the best of the three algorithms.